MICROGOALS – SECURE GOAL MANAGEMENT BACKEND API
(Naan Mudhalvan Project Documentation)

Introduction

• Team ID: NM2025TMID36488 • Team Leader: Rathipriya P & 24csrathipriya077@gmail.com • Team Members: – Rathika K & 24csrathika076@gmail.com – Vaishnavi S & 24csvaishnavi106@gmail.com – Sujithra G & 24cssujithra100@gmail.com

1. Project Overview

MicroGoals is a secure backend RESTful API developed for managing personal and professional goals. The system allows users to register, authenticate themselves, and manage their goals through operations like creating, updating, viewing, and deleting goals. This project is developed using Node.js, Express.js, and MongoDB. It follows modern backend development practices such as REST API design, JWT authentication, MVC architecture, and middleware-based security implementation. The main purpose of this system is to provide a secure digital goal management platform where users can track their goals effectively. Each user can maintain their own goals and track their completion progress. MicroGoals demonstrates how backend APIs can securely manage user data, perform CRUD operations, and implement authentication mechanisms.

2. Project Instruction

- The system is designed based on REST API architecture
- JWT authentication is used to secure APIs
- MongoDB database is used for storing data
- Thunder Client / Postman is used to test APIs
- MVC architecture is implemented for structured development
- Middleware is used for authentication and error handling

3. Pre-requisites

Software Requirements

- Node.js (version 18 or above)
- MongoDB
- Visual Studio Code
- Thunder Client or Postman
- Web Browser
-

Knowledge Requirements

- Basic JavaScript
- REST API concepts
- MongoDB basics
- Backend development fundamentals

4. Project Architecture

Architecture Components: User → Frontend Application → REST API Server (Node.js + Express.js) → Authentication Layer (JWT Middleware) → Database Layer (MongoDB).

5. Technical Architecture

- Runtime Environment: Node.js
- Framework: Express.js
- Database: MongoDB
- Authentication: JWT
- Password Encryption: bcryptjs
- API Testing: Thunder Client
- Architecture Pattern: MVC

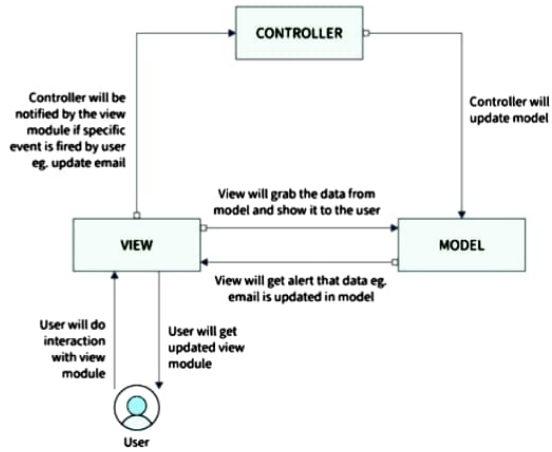
6. ER Diagram

Entities: User and Goal.

User attributes: _id, name, email, password.

Goal attributes: _id, title, completed, userId.

Relationship: One User can have many Goals.



7. Features

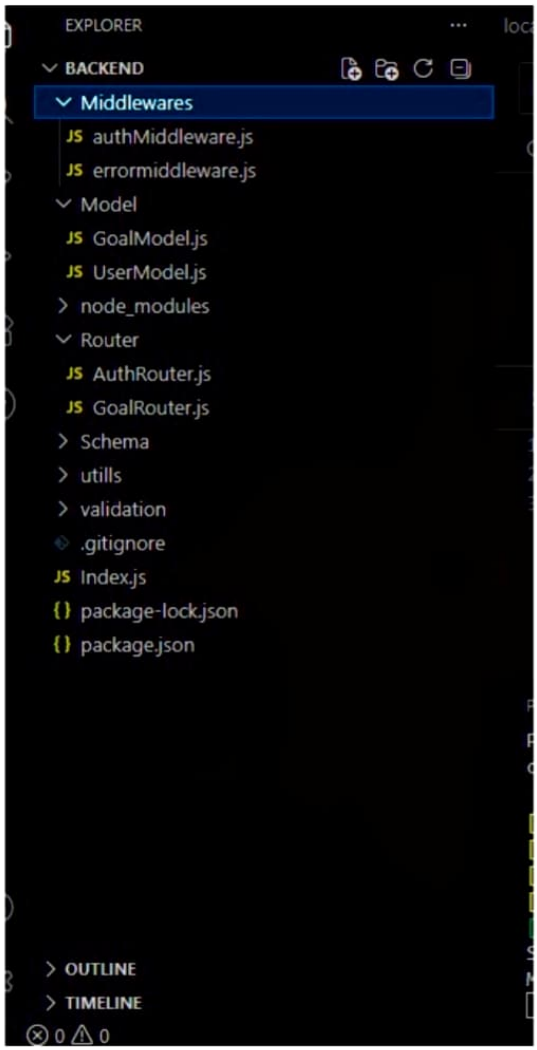
User Management

- User registration
- User login
- JWT token generation

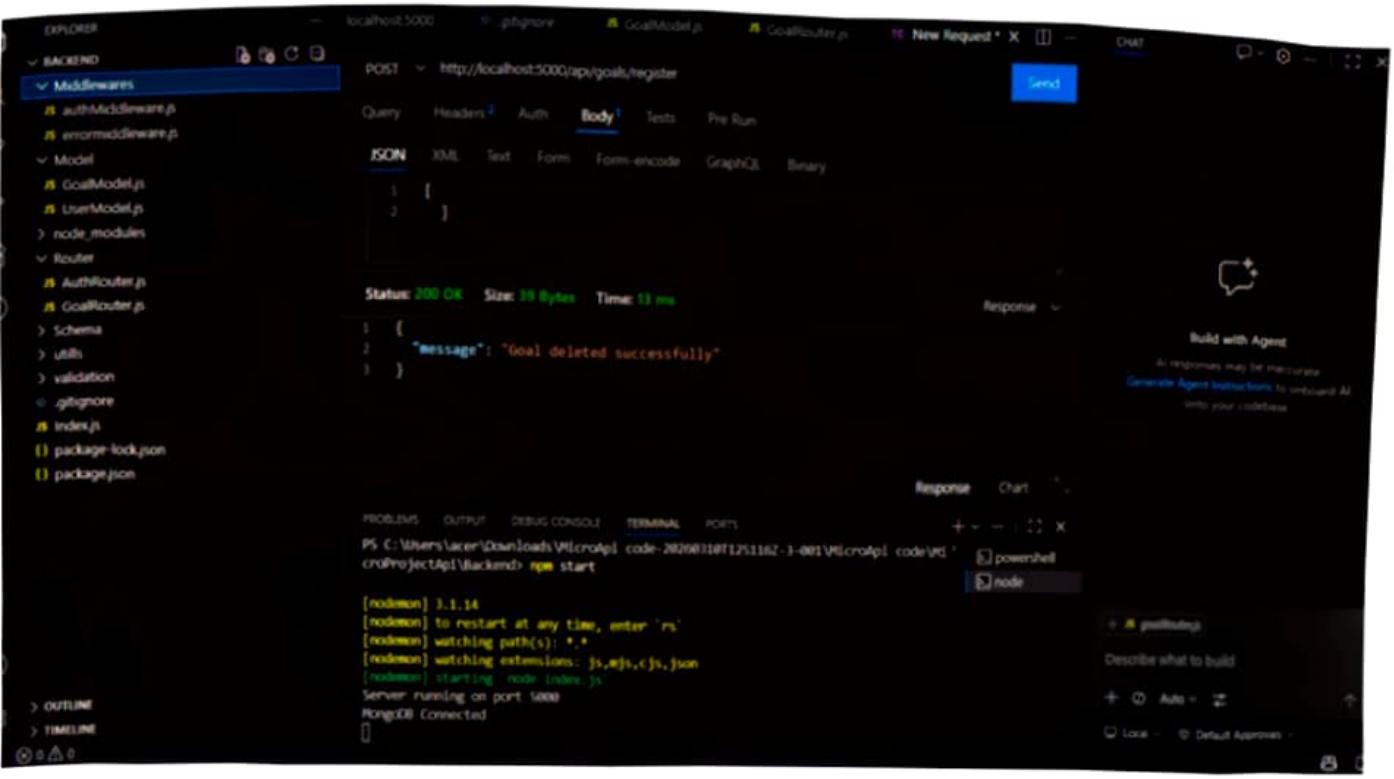
Goal Management

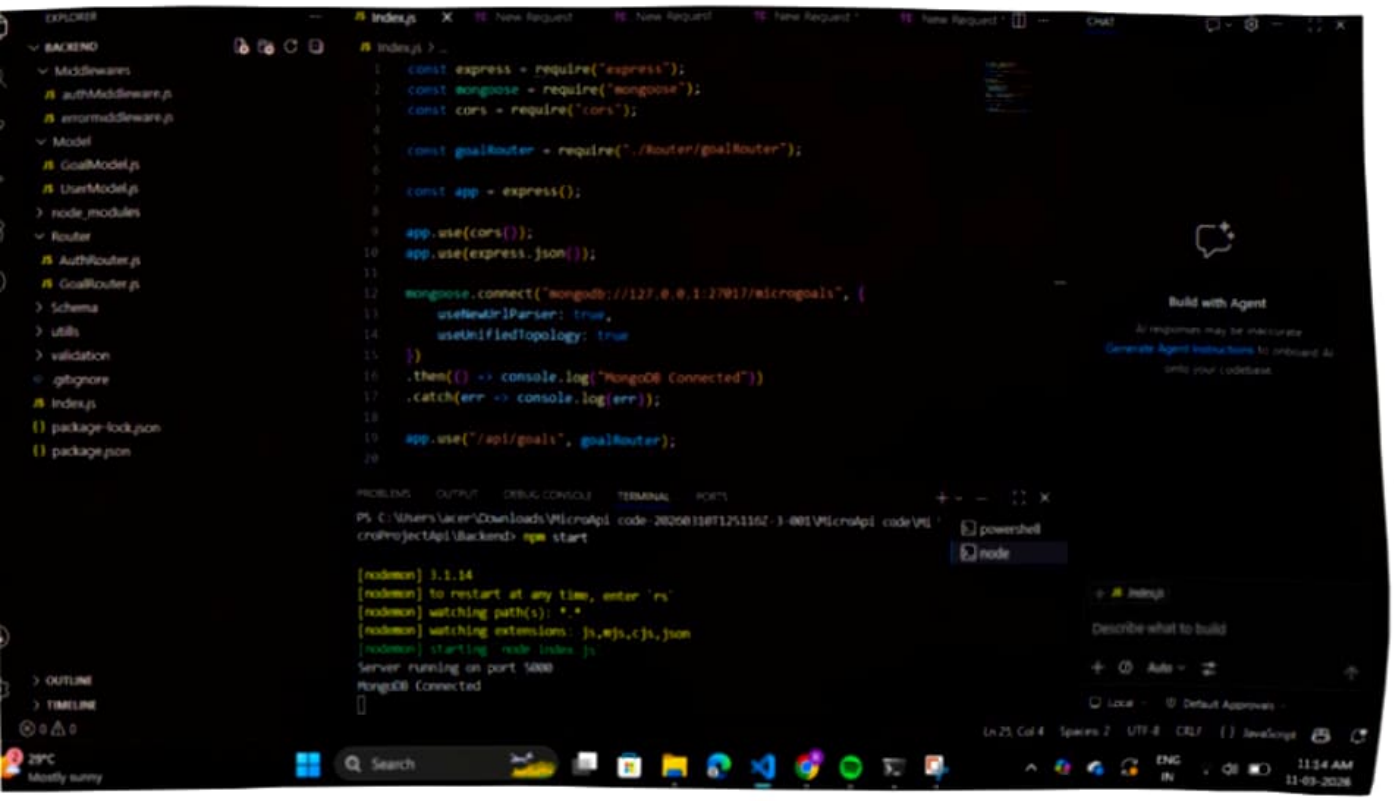
- Create goals

elements used



enshots of our project





- View goals
- Update goals
- Delete goals

Security Features

- JWT authentication
- Password hashing
- Protected routes
- Error handling middleware

8. Project Setup

```
npm init -y
npm install express mongoose jsonwebtoken bcryptjs dotenv
node index.js
```

9. Database Development

MongoDB is used to store application data. It is a NoSQL document database that stores data in JSON-like format.

- Flexible schema
- High performance
- Easy scalability
- JSON document structure

10. Configure MongoDB

```
MONGO_URI = mongodb://localhost:27017/microgoals
```

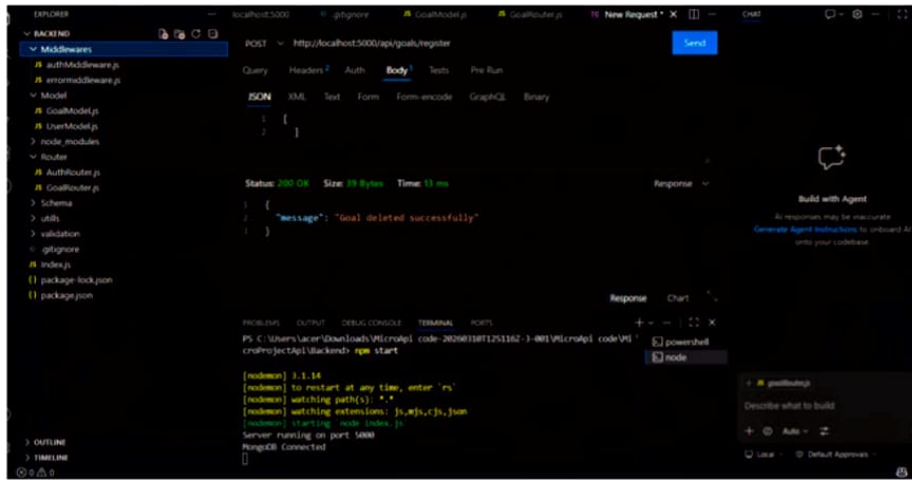
11. Database Connection Example

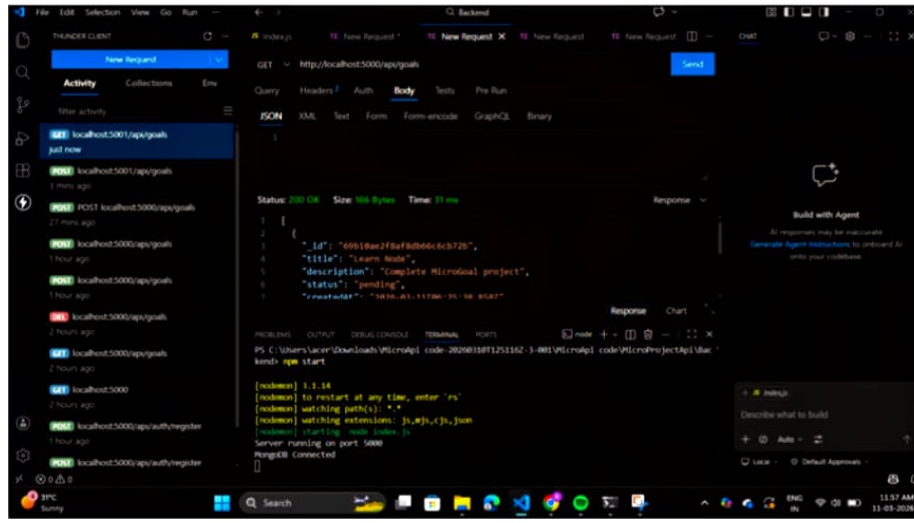
```
mongoose.connect(process.env.MONGO_URI)
.then(()=> console.log("Database Connected"))
.catch(err => console.log(err));
```

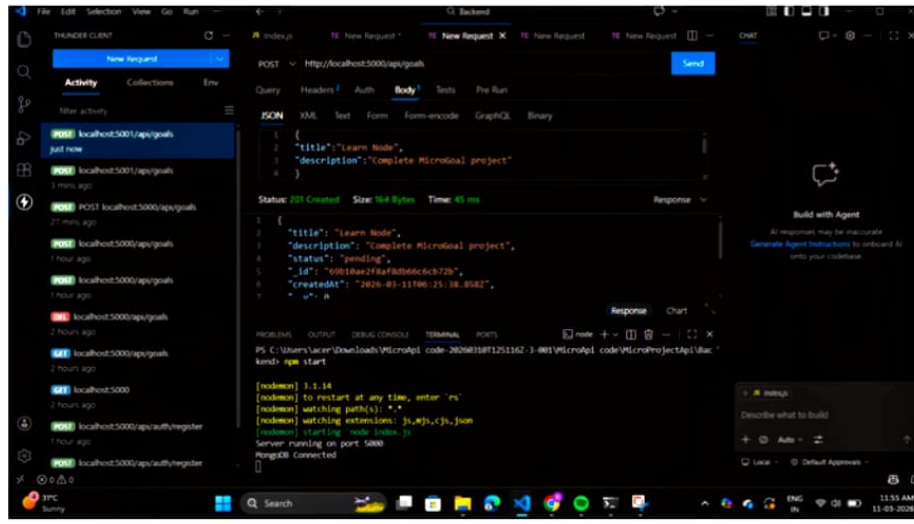
12. Advantages

- Secure authentication system
- Scalable backend architecture
- Efficient goal management
- Easy integration with frontend

operation used in thunder client







- Organized code structure

13. Limitations

- No frontend interface included
- Basic goal management features
- Limited analytics for goal progress

14. Future Enhancements

- Mobile application integration
- Goal reminder notifications
- Progress tracking charts
- Deadline based goals
- Social goal sharing

Dongobd connection

15. Conclusion

MicroGoals – Secure Goal Management Backend API is a well-structured backend system developed using Node.js, Express.js, and MongoDB. It demonstrates how secure APIs can be built using modern web technologies. The project implements authentication, database management, and CRUD operations for managing user goals and provides a strong foundation for scalable backend applications.